Cache Invalidation

Cache Invalidation Strategies

While caching is fantastic, it requires some maintenance to keep the cache coherent with the source of truth (e.g., database). If the data is modified in the database, it should be invalidated in the cache; if not, this can cause inconsistent application behavior.

Solving this problem is known as cache invalidation; there are four main schemes that are used:

1. Write-through cache

Under this scheme, data is written into the cache and the corresponding database simultaneously. The cached data allows for fast retrieval and, since the same data gets written in the permanent storage, we will have complete data consistency between the cache and the storage. Also, this scheme ensures that nothing will get lost in case of a crash, power failure, or other system disruptions. Although, write-through minimizes the risk of data loss, since every write operation must be done twice before returning success to the client, this scheme has the disadvantage of higher latency for write operations.

2. Write-around cache

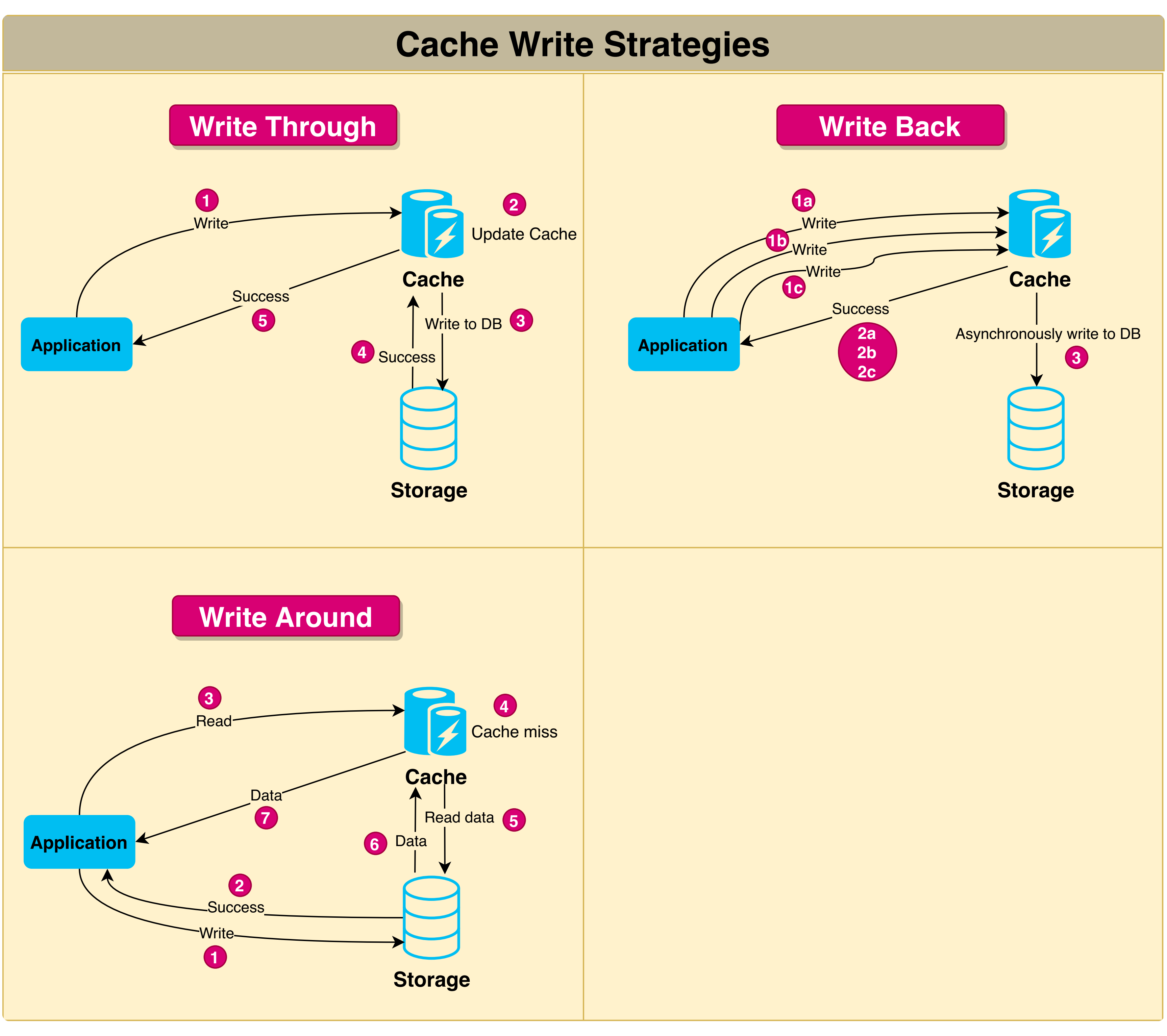
This technique is similar to write-through cache, but data is written directly to permanent storage, bypassing the cache. This can reduce the cache being flooded with write operations that will not subsequently be re-read, but has the disadvantage that a read request for recently written data will create a “cache miss” and must be read from slower back-end storage and experience higher latency.

3. Write-back cache

Under this scheme, data is written to cache alone, and completion is immediately confirmed to the client. The write to the permanent storage is done based on certain conditions, for example, when the system needs some free space. This results in low-latency and high-throughput for write-intensive applications; however, this speed comes with the risk of data loss in case of a crash or other adverse event because the only copy of the written data is in the cache.

4. Write-behind cache

It is quite similar to write-back cache. In this scheme, data is written to the cache and acknowledged to the application immediately, but it is not immediately written to the permanent storage. Instead, the write operation is deferred, and the data is eventually written to the permanent storage at a later time. The main difference between write-back cache and write-behind cache is when the data is written to the permanent storage. In write-back caching, data is only written to the permanent storage when it is necessary for the cache to free up space, while in write-behind caching, data is written to the permanent storage at specified intervals.


Cache Write Strategies
Write Through
1 Write
2 Update Cache
Cache
3 Write to DB
4 Success
5 Success
Application
Storage
Write Back
1a Write
1b Write
1c Write
Success
2a 2b 2c
Cache
Asynchronously write to DB
3
Application
Storage
Write Around
3 Read
4 Cache miss
Cache
Data 7
Read data 5
6 Data
Application
2 Success
Write 1
Storage

Cache Invalidations Methods

Here are the famous cache invalidation methods:

Purge

The purge method removes cached content for a specific object, URL, or a set of URLs. It’s typically used when there is an update or change to the content and the cached version is no longer valid. When a purge request is received, the cached content is immediately removed, and the next request for the content will be served directly from the origin server.

Refresh

Fetches requested content from the origin server, even if cached content is available. When a refresh request is received, the cached content is updated with the latest version from the origin server, ensuring that the content is up-to-date. Unlike a purge, a refresh request doesn’t remove the existing cached content; instead, it updates it with the latest version.

Ban

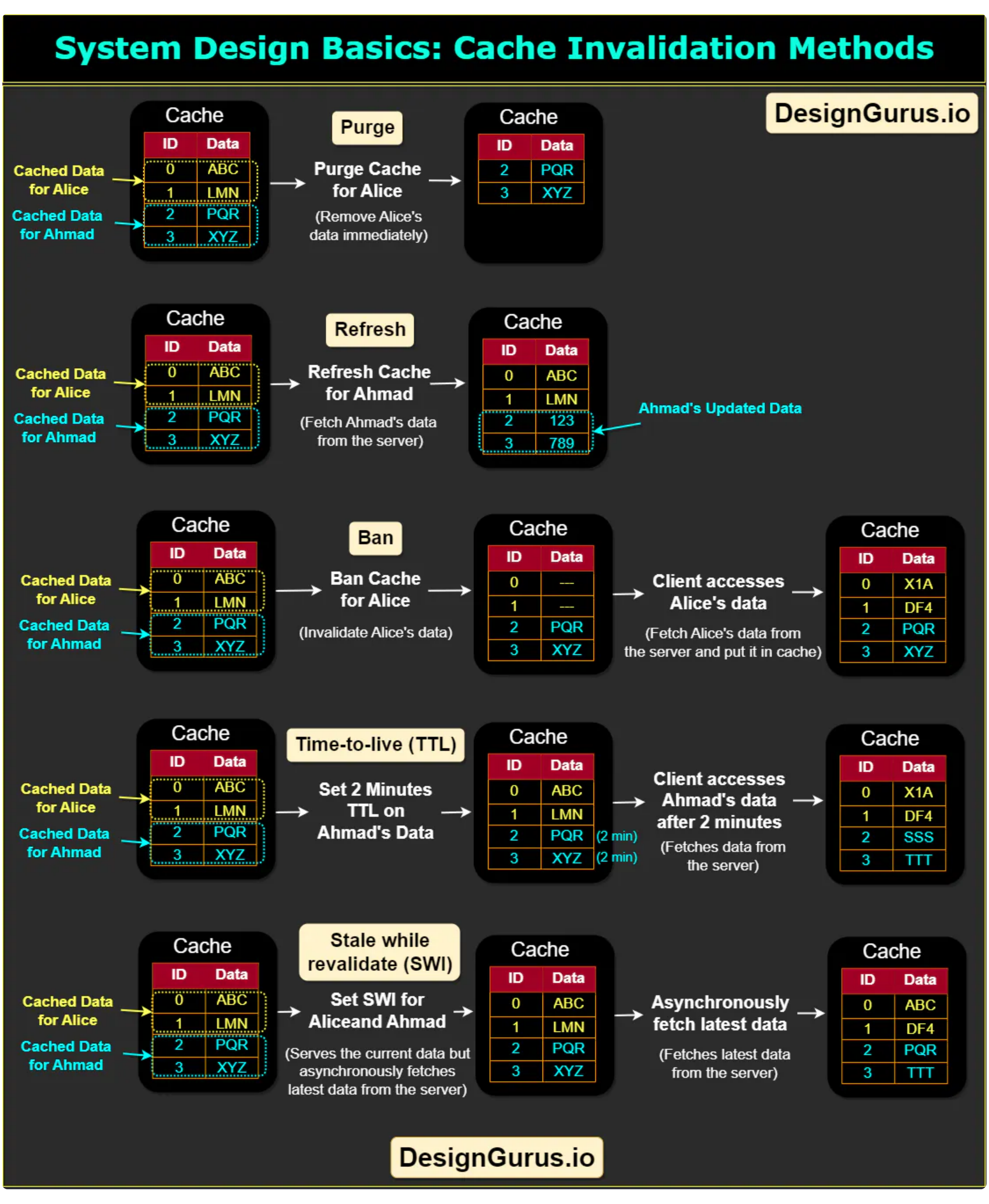
The ban method invalidates cached content based on specific criteria, such as a URL pattern or header. When a ban request is received, any cached content that matches the specified criteria is immediately removed, and subsequent requests for the content will be served directly from the origin server.

Time-to-live (TTL) expiration

This method involves setting a time-to-live value for cached content, after which the content is considered stale and must be refreshed. When a request is received for the content, the cache checks the time-to-live value and serves the cached content only if the value hasn’t expired. If the value has expired, the cache fetches the latest version of the content from the origin server and caches it.

Stale-while-revalidate

This method is used in web browsers and CDNs to serve stale content from the cache while the content is being updated in the background. When a request is received for a piece of content, the cached version is immediately served to the user, and an asynchronous request is made to the origin server to fetch the latest version of the content. Once the latest version is available, the cached version is updated. This method ensures that the user is always served content quickly, even if the cached version is slightly outdated.


System Design Basics: Cache Invalidation Methods
DesignGurus.io
Cached Data for Alice
Cached Data for Ahmad
Cache
ID Data
0 ABC
1 LMN
2 PQR
3 XYZ
Purge
Purge Cache for Alice
(Remove Alice's data immediately)
Cache
ID Data
2 PQR
3 XYZ
Refresh
Refresh Cache for Ahmad
(Fetch Ahmad's data from the server)
Cache
ID Data
0 ABC
1 LMN
2 123
3 789
Ahmad's Updated Data
Ban
Ban Cache for Alice
(Invalidate Alice's data)
Cache
ID Data
0 ---
1 ---
2 PQR
3 XYZ
Client accesses Alice's data
(Fetch Alice's data from the server and put it in cache)
Cache
ID Data
0 X1A
1 DF4
2 PQR
3 XYZ
Time-to-live (TTL)
Set 2 Minutes TTL on Ahmad's Data
Cache
ID Data
0 ABC
1 LMN
2 PQR (2 min)
3 XYZ (2 min)
Client accesses Ahmad's data after 2 minutes
(Fetches data from the server)
Cache
ID Data
0 X1A
1 DF4
2 SSS
3 TTT
Stale while revalidate (SWI)
Set SWI for Aliceand Ahmad
(Serves the current data but asynchronously fetches latest data from the server)
Cache
ID Data
0 ABC
1 LMN
2 PQR
3 XYZ
Asynchronously fetch latest data
(Fetches latest data from the server)
Cache
ID Data
0 ABC
1 DF4
2 PQR
3 TTT
DesignGurus.io

← Previous
Cache Replacement Policies

Next →
Cache Read Strategies

☐ Mark as Completed